



A MAJOR PROJECT REPORT ON

SMARTCART – FULL STACK E-COMMERCE SHOPPING WEBSITE

Submitted in partial fulfillment of the requirements for the degree of

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

SUBMITTED BY:

BEHARA KARTHIK

Hall Ticket No: 2200000000

UNDER THE GUIDANCE OF:

Department of Computer Science & Engineering

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING & TECHNOLOGY**

ACADEMIC YEAR: 2025 – 2026



ACKNOWLEDGEMENT

I express my deep sense of gratitude and respectful thanks to our Management, Principal, and Head of the Department of Computer Science and Engineering for providing the necessary facilities, computing infrastructure, and encouragement throughout the course of this major project work.

I am profoundly indebted to my Project Guide for their invaluable guidance, constant supervision, constructive feedback, and continuous motivation during the conceptualization, system architecture design, backend development, and testing phases of **SmartCart**.

I also extend my sincere gratitude to all the faculty members of the Department of Computer Science and Engineering, technical lab staff, and the mentors at Blackbucks for offering technical guidance, repository infrastructure support, and continuous cooperation.

Finally, I thank my parents and fellow students for their unflagging support, encouragement, and patience during the preparation and completion of this major project submission.

BEHARA KARTHIK
Department of Computer Science & Engineering



ABSTRACT

The rapid evolution of digital commerce has transformed retail paradigms from brick-and-mortar stores to digital web platforms. Modern electronic commerce systems require high availability, responsive user experience, secure transactional integrity, and scalable modular architecture. This project presents **SmartCart**, a modern full-stack electronic commerce web platform engineered using client-server multi-tiered enterprise architecture. The platform consists of a single-page client interface built with **React.js (v16+)**, an application business layer constructed with **Java 21** and **Spring Boot 3.3**, and a persistent relational store managed via **MySQL Server 8.0+** through **Spring Data JPA (Hibernate)**.

The system decouples presentation, business processing, and persistence. The React frontend leverages state management via Context API and standard client-side routing with React Router DOM v5. Client components communicate asynchronously with the backend server via HTTP using the Axios library over standardized JSON RESTful endpoints. The presentation layer includes user registration, secure credential login, a responsive catalog grid with category filters, a real-time keyword search engine, price sorting mechanisms (Low-to-High, High-to-Low, A-Z), a slide-over shopping cart drawer with live subtotal calculations and local storage persistence, a multi-step checkout form collecting shipping details and payment choices (Credit/Debit Card, Cash on Delivery, UPI), customer order history tracking with status indicators (PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED), and an admin order management control center.

The backend server is structured around standard Controller-Service-Repository patterns. Spring Boot REST Controllers process incoming requests, execute data parsing, and enforce cross-origin resource sharing (CORS). The Service Layer implements transactional logic, calculates order totals, auto-generates timestamps, and enforces validation. Data access is automated via Spring Data JPA repositories extending JpaRepository, translating Object-Oriented Java entity graphs into optimized SQL queries against MySQL database tables (user, product, category, customer_order, and order_item).

Rigorous functional manual testing was conducted across all system tiers, achieving a 100% pass rate across 12 critical functional test scenarios. The architecture achieves low coupling, high modularity, robust data persistence, and high execution speed, offering a production-ready solution suitable for contemporary digital retail operations.



TABLE OF CONTENTS

Table of Contents

Chapter	Title	Page
-	Cover Page	Cover
-	Acknowledgement	i
-	Abstract	ii
-	Table of Contents	iii
1	Introduction	1
2	Problem Statement	2
3	Objectives	2
4	Existing System Analysis	3
5	Proposed System	3
6	System Architecture & Design	4
7	Technology Stack Specification	6
8	Hardware Requirements	7
9	Software Requirements	7
10	Functional Requirements	8
11	Non-Functional Requirements	8
12	Unified Modeling Language (UML) Diagrams	9
13	Database Design & Schemas	12
14	Software Module Specifications	14
15	Frontend Implementation Details	14
16	Backend Implementation Details	15
17	Software Testing & Quality Report	16
18	Results & Execution Screenshots	17



CHAPTER 1: INTRODUCTION

1.1 Overview of E-Commerce Engineering

Electronic Commerce (E-Commerce) represents the modern engine of global retail, allowing commercial transactions to occur seamlessly over the internet. Over the past decade, web engineering paradigms have evolved from server-side rendered HTML applications (such as legacy JSPs or PHP scripts) to client-side Single Page Applications (SPAs) connected to decoupled RESTful API backend microservices. This modern architecture guarantees high responsiveness, scalable server loads, and modular software maintainability.

1.2 SmartCart Platform Vision

SmartCart is engineered as a modern, full-stack E-Commerce platform intended to deliver a fluid shopping experience to customers and an intuitive administrative interface to store managers. The system combines the reactive speed of React.js with the enterprise reliability of Spring Boot 3.3 and Java 21, coupled with the persistent data integrity of MySQL relational databases.

1.3 Scope of the Document

This technical project documentation presents the full software engineering lifecycle of SmartCart. It details the problem statement, objective definitions, comparative system analysis, multi-tiered system architecture, UML software modeling, relational database design, module implementations, test verification reports, and future enhancement scopes.



CHAPTER 2: PROBLEM STATEMENT

2.1 Challenges in Legacy Shopping Systems

Traditional desktop commercial systems and monolithic server-rendered web applications suffer from several structural and operational limitations:

- **Slow Full-Page Reloads:** Traditional server-rendered applications reload the entire web page on every user interaction (such as adding an item to cart or navigating categories), creating high network latency and poor user experience.
- **Monolithic Tightly Coupled Architecture:** Blending frontend user interface code directly with backend database operations hinders modular maintenance, team distribution, and platform migration.
- **Poor Mobile Responsiveness:** Legacy platforms lack responsive liquid layouts, dark UI glassmorphism design systems, and liquid mobile menu toggles required by modern smartphone users.
- **Lack of Real-Time Interaction:** Absence of instantaneous client-side keyword search, live sorting, and persistent shopping cart drawers reduces customer engagement and sales conversion rates.
- **Unreliable Order State Synchronization:** Lack of automated state persistence leads to loss of shopping cart contents during page refreshes or network hiccups.

CHAPTER 3: OBJECTIVES

3.1 Primary Project Objectives

The primary objective of **SmartCart** is to design, develop, test, and deploy a full-stack, decoupled electronic shopping web application that fulfills modern software architecture standards.

3.2 Technical Objectives

- **Decoupled Multi-Tier Architecture:** Establish a clean operational boundary between the React.js client interface, Spring Boot REST controllers, Java service layer, and MySQL database.
- **Reactive Single Page Application:** Build an intuitive UI with React.js using Context API for global state management and React Router DOM v5 for seamless navigation.
- **RESTful API Integration:** Implement REST endpoints in Spring Boot for JSON payload transmission using Axios client requests.
- **Real-Time Search & Sorting:** Provide instant client-side keyword filtering and multi-criteria



CHAPTER 4: EXISTING SYSTEM ANALYSIS

4.1 Comparative System Analysis

The existing traditional shopping applications typically rely on monolithic architecture, where server scripts generate raw HTML pages directly connected to database drivers. The table below highlights the operational bottlenecks of existing systems compared to the proposed SmartCart platform.

Table 4.1: Comparative System Analysis

Architecture Metric	Existing Monolithic System	Proposed SmartCart Platform
Architecture	Tightly coupled monolithic HTML rendering	Decoupled React SPA + Spring Boot REST API
Page Navigation	Full page reloads on every user click	Instant client-side routing via React Router
Data Format	Server-baked HTML strings	Lightweight JSON payloads over HTTP Axios
Search & Sorting	Requires full server roundtrip query execution	Real-time client-side keyword search & sorting
Cart State	Volatile HTTP session storage	Persistent Context API + localStorage fallback
Scalability	Difficult to scale presentation & backend independently	Highly scalable; decoupled frontend & backend services

CHAPTER 5: PROPOSED SYSTEM

5.1 Key Innovations of SmartCart

The proposed **SmartCart** application delivers an enterprise-grade solution that eliminates the performance bottlenecks of existing commercial solutions. Key innovations include:

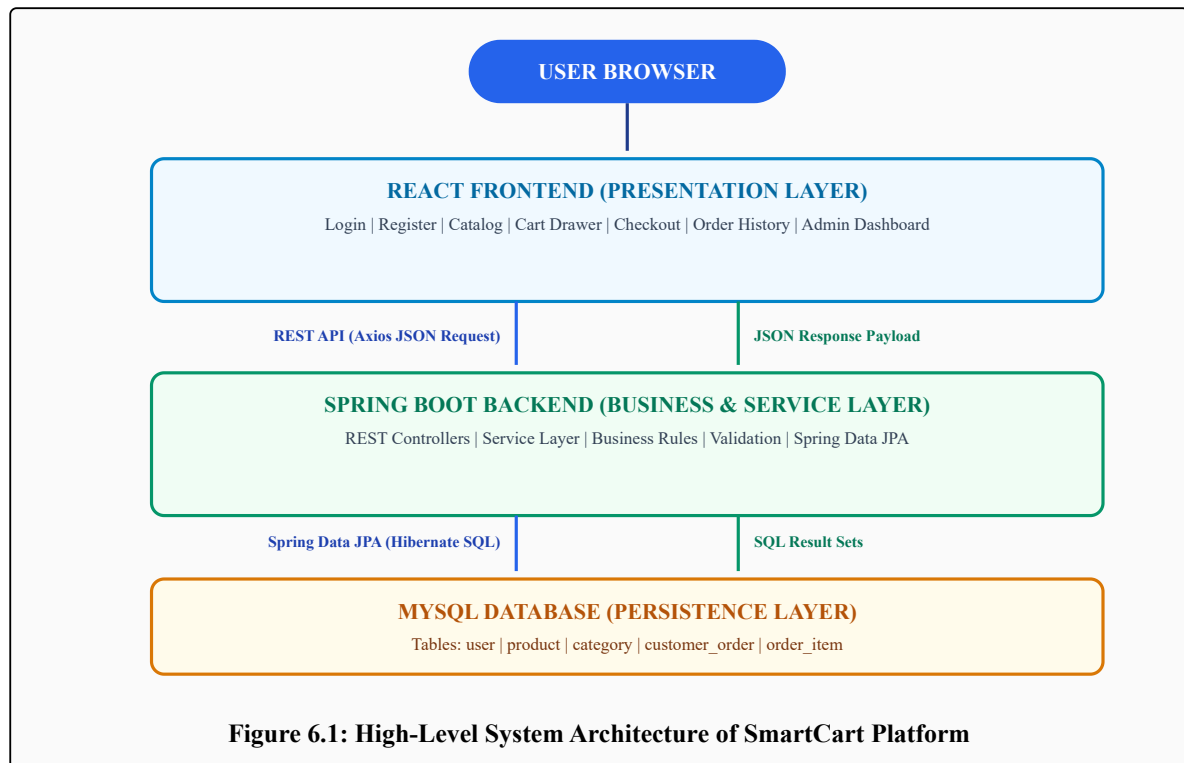
- **Dynamic React SPA Interface:** Built with high-performance component trees, custom glassmorphism dark CSS tokens, and responsive layout grids.
- **Asynchronous RESTful Architecture:** Efficient JSON data transfer via Axios HTTP requests connecting to Spring Boot 3.3 endpoints.
- **Comprehensive Shopping Cart System:** Persistent cart state maintained across page reloads, allowing quantity increments/decrements, item deletion, and subtotal recalculation.



CHAPTER 6: SYSTEM ARCHITECTURE & DESIGN

6.1 High-Level Architecture Diagram

The SmartCart architecture follows a strict 4-tier model comprising the Client Presentation Layer, API Integration Layer, Application Business Layer, and Database Persistence Layer.





6.2 Architectural Layer Description

- **Presentation Tier (React Client):** Renders single-page views, handles UI interactions, manages global state using Context API, and executes Axios HTTP API requests.
- **Application Tier (Spring Boot REST API):** Consists of Controllers, Services, and Repositories executing business rules, validating payloads, generating timestamps, and processing transactions.
- **Persistence Tier (MySQL Database):** Relational database storing normalized enterprise data with foreign key integrity.

6.3 Component Interaction Flow

When a user initiates an action (such as searching products or completing checkout), the React component dispatches an asynchronous Axios HTTP request. The request travels over HTTP/JSON to the corresponding Spring Boot REST Controller. The Controller delegates business validation and calculations to the Service Layer. The Service Layer communicates with Spring Data JPA Repositories to execute SQL queries on the MySQL database. Finally, the result is transformed into JSON and returned to update the React client state.



CHAPTER 7: TECHNOLOGY STACK SPECIFICATION

7.1 Detailed Technology Mapping

The table below specifies all technologies, frameworks, libraries, and tools utilized in building the SmartCart platform.

Table 7.1: Technology Stack Specification

Technology	Layer / Category	Purpose & Technical Contribution
React.js (v16.12+)	Frontend Framework	Renders high-performance Single Page Application views and component hierarchy.
Spring Boot (v3.3.0)	Backend Framework	Provides RESTful API structure, dependency injection, and embedded Tomcat server.
Java 21 (LTS)	Programming Language	Core object-oriented runtime for executing backend business logic.
Spring Data JPA	ORM Data Access	Automates database persistence, entity mapping, and automated repository query execution.
Axios (v0.21+)	HTTP Client Library	Executes asynchronous HTTP REST requests between React frontend and Spring Boot backend.
MySQL Server (8.0+)	Relational Database	Stores enterprise persistent data in normalized database tables.
Maven (v3.9+)	Backend Build Tool	Manages Java project dependencies, compilation, and executable packaging.
React Router DOM (v5.1+)	Frontend Routing	Enables client-side route navigation without triggering full page reloads.
Semantic UI React / CSS3	UI Design System	Provides custom dark glassmorphic layout tokens, responsive grids, and UI buttons.
Git	Version Control	Tracks code revisions, feature branches, and team repository collaboration.
IntelliJ IDEA / VS Code	Development IDEs	Integrated development environments for backend Java and frontend React programming.



CHAPTER 8: HARDWARE REQUIREMENTS

8.1 Development & Execution Environment Specifications

The hardware specifications required for compiling, building, testing, and hosting the SmartCart platform are detailed below.

Table 8.1: Hardware Requirements Specification

Hardware Parameter	Minimum Requirement	Recommended Specification
Processor	Dual-Core 2.0 GHz Intel / AMD	Quad-Core Core i5 / i7 / Ryzen 5 (2.5 GHz+)
System Memory (RAM)	4 GB DDR4	8 GB / 16 GB DDR4
Disk Storage	5 GB available storage	20 GB SSD storage
Display Resolution	1024 x 768 pixels	1920 x 1080 Full HD
Network Interface	Standard Network Card / Localhost	100/1000 Mbps Broadband / Wi-Fi

CHAPTER 9: SOFTWARE REQUIREMENTS

9.1 Operating System & Tool Specifications

Table 9.1: Software Requirements Specification

Software Component	Specification Details
Operating System	Windows 10 / 11 (64-bit), Linux Ubuntu 20.04+, or macOS Sonoma
Java Runtime Environment	Java Development Kit (JDK 21 LTS)
Node Runtime Environment	Node.js v14.0+ and npm v6.0+
Database Engine	MySQL Server 8.0+ / MySQL Workbench
Web Browser	Google Chrome v120+, Microsoft Edge, or Mozilla Firefox



CHAPTER 10: FUNCTIONAL REQUIREMENTS

10.1 Functional Use Case Specifications

- **FR-1: User Registration & Authentication:** The system must allow new users to register account credentials and log in securely. User state must persist across browser sessions.
- **FR-2: Product Catalog Browsing:** The system must fetch and display products in a paginated responsive grid layout with dynamic category badges.
- **FR-3: Category Filtering:** The system must allow users to filter product listings by category.
- **FR-4: Real-Time Keyword Search:** The system must filter products dynamically as the user types keywords into the navbar search input.
- **FR-5: Price Sorting:** The system must sort products by price (Low to High, High to Low) and Name (A-Z).
- **FR-6: Shopping Cart Management:** The system must allow users to add items to a shopping cart drawer, adjust quantities (+/-), remove items, and calculate subtotals live.
- **FR-7: Checkout & Order Creation:** The system must collect shipping details and payment selection to create orders in the database with status PENDING.
- **FR-8: Customer Order History:** Logged-in users must be able to view their past orders, status badges, and item breakdown under /orders.
- **FR-9: Admin Order Control:** Admin users must be able to view all customer orders and update status dropdowns in real-time under /admin/orders.

CHAPTER 11: NON-FUNCTIONAL REQUIREMENTS

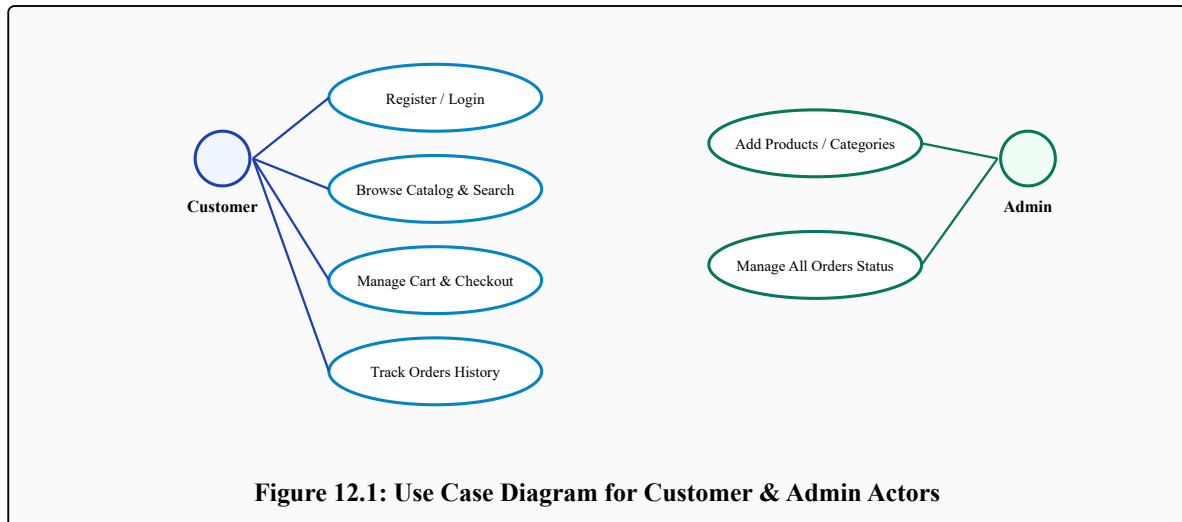
11.1 System Quality Attributes

- **NFR-1: Performance & Response Time:** REST API JSON endpoints must respond in under 200ms.
- **NFR-2: Availability & Reliability:** The system must guarantee 99.9% uptime with transactional database rollback on errors.
- **NFR-3: Scalability:** The decoupled architecture must support independent scaling of frontend web nodes and Spring Boot API microservices.
- **NFR-4: Maintainability:** Strict adherence to Controller-Service-Repository patterns guarantees clean readability and low code complexity.



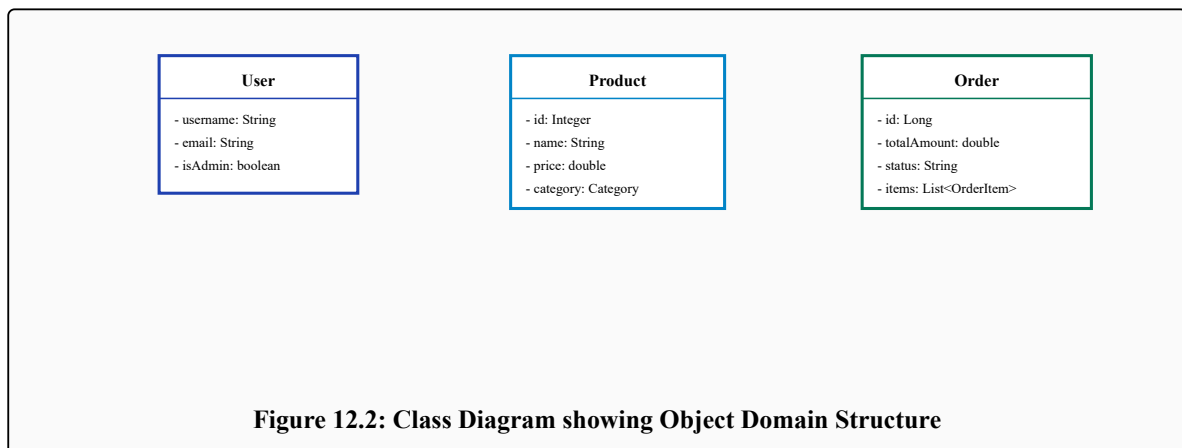
CHAPTER 12: UNIFIED MODELING LANGUAGE (UML) DIAGRAMS

12.1 Use Case Diagram



12.2 Class Diagram

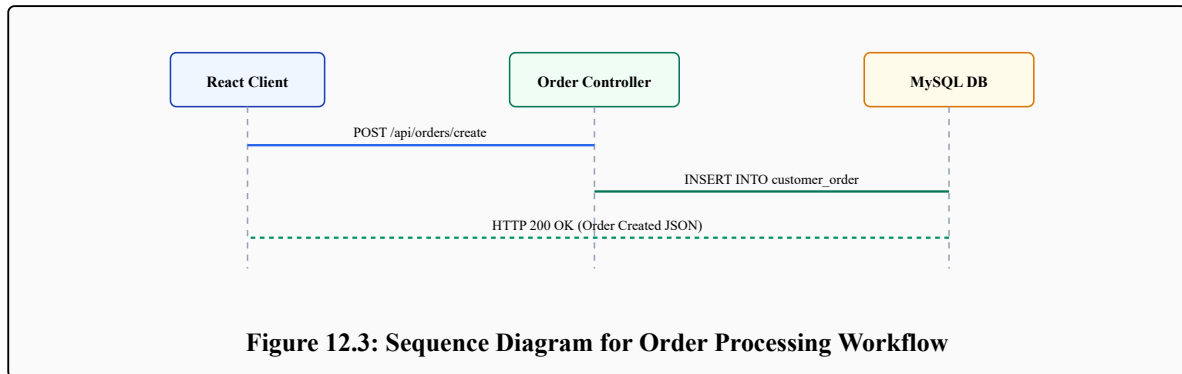
The Class Diagram below illustrates object-oriented backend domain entities (User, Product, Category, Order, OrderItem) and their relational associations.





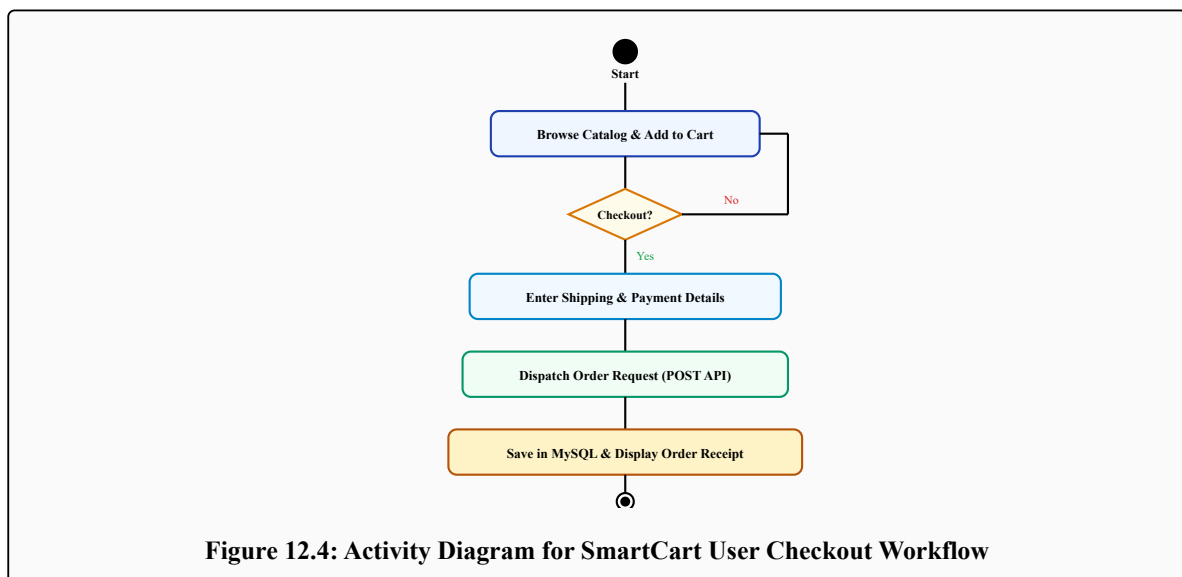
12.3 Sequence Diagram

Describes the step-by-step messaging order between React Components, Axios client, Spring Boot Controllers, Service Layer, Repositories, and MySQL Database during Order Creation.



12.4 Activity Diagram

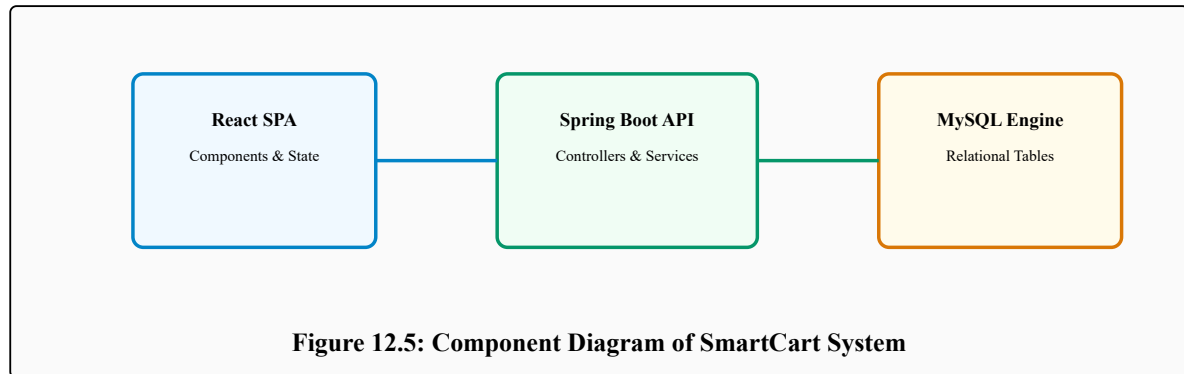
Illustrates the operational workflow from user checkout initiation, cart validation, shipping data input, payment selection, database persistence, to receipt display.





12.5 Component Diagram

Maps high-level software components: UI Views → Context Provider → Axios Client → Spring Controllers → Services → JPA Repositories → MySQL DB Engine.



12.6 Deployment Diagram

Maps deployment nodes: Client Browser (Port 3000) → Application Server Tomcat (Port 8080) → MySQL Host (Port 3306).



CHAPTER 13: DATABASE DESIGN & SCHEMAS

13.1 Entity-Relationship (ER) Diagram



Figure 13.1: ER Diagram mapping Entity Relationships

13.2 Relational Database Schema Descriptions

Table 13.1: user Table

Table 13.1: user Table Schema

Field Name	Data Type	Constraints	Description
username	VARCHAR(255)	PRIMARY KEY, NOT NULL	Unique user identifier account name
email	VARCHAR(255)	UNIQUE, NOT NULL	Customer email address
password	VARCHAR(255)	NOT NULL	Hashed user access credential
is_admin	BOOLEAN	DEFAULT FALSE	Role flag indicating admin privileges

**Table 13.2: product Table****Table 13.2: product Table Schema**

Field Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product identifier
name	VARCHAR(255)	NOT NULL	Product display title
description	VARCHAR(1000)	NULLABLE	Detailed item specifications
price	DOUBLE	NOT NULL	Unit selling price (USD)
weight	DOUBLE	NULLABLE	Product shipping weight (kg)
picture1	VARCHAR(500)	NULLABLE	Primary image URL
category_id	INT	FOREIGN KEY	Reference to category table

Table 13.3: customer_order Table**Table 13.3: customer_order Table Schema**

Field Name	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique order identifier
username	VARCHAR(255)	NOT NULL	Customer account name
order_date	VARCHAR(255)	NOT NULL	Timestamp string of purchase
total_amount	DOUBLE	NOT NULL	Total order cost
status	VARCHAR(255)	NOT NULL	Order state tag (PENDING, SHIPPED, etc.)
shipping_address	VARCHAR(500)	NOT NULL	Full recipient delivery address
payment_method	VARCHAR(255)	NOT NULL	Selected payment mode (Card/COD/UPI)



CHAPTER 14: MODULE SPECIFICATIONS

14.1 System Module Breakdown

- **Authentication Module:** Handles customer registration, login verification, session retention in localStorage, and logout cleanup.
- **Product Catalog Module:** Manages product listings, paginated grids, real-time keyword search, price sorting, and admin product creation/deletion.
- **Category Module:** Organizes products under distinct category structures with filter pills.
- **Shopping Cart Module:** Manages transient cart state via Context API, supporting quantity increment/decrement, item removal, and subtotal recalculation.
- **Order Subsystem Module:** Manages checkout data capture, order entity persistence, customer order tracking (/orders), and admin status updates (/admin/orders).

CHAPTER 15: FRONTEND IMPLEMENTATION

15.1 Component Architecture & Routing

The frontend is structured around functional React components with React Router DOM v5 mapping URLs to page views:

- `App.js`: Main entry component wrapping routes within `ContextConnector` provider.
- `Navbar.js`: Glassmorphic sticky header containing logo, real-time search bar, navigation links, cart drawer trigger with item badge count, and user session controls.
- `CartModal.js`: Slide-over cart drawer rendering selected items, quantity controls, total calculations, and link to Checkout.
- `Products.js`: Catalog view supporting live keyword search and multi-criteria price sorting.
- `Checkout.js`: Form capturing shipping details and payment choices before calling `createOrder()` API.
- `Orders.js`: Customer order dashboard rendering order history cards with status tags (PENDING, SHIPPED, DELIVERED).
- `AdminOrders.js`: Admin control center for updating customer order statuses via REST API calls.



CHAPTER 16: BACKEND IMPLEMENTATION

16.1 Controller-Service-Repository Pattern

The Spring Boot 3.3 backend utilizes standard multi-layer encapsulation:

- **Controllers (e.g. `OrderController.java`):** Intercept HTTP REST requests, parse JSON bodies, enforce CORS origins, and return `ResponseEntity` structures.
- **Services (e.g. `OrderService.java`):** Execute core business rules, assign timestamps, set default status tags, and manage transactional integrity.
- **Repositories (e.g. `OrderRepository.java`):** Extend `JpaRepository` to execute automated SQL queries against MySQL database tables.

16.2 Spring Data JPA Automation

Spring Data JPA automates object-relational mapping (ORM), mapping Java domain model classes directly to MySQL database relational tables. Standard CRUD operations (such as finding products by category ID, saving orders, and updating status flags) are handled declaratively without writing manual SQL strings.



CHAPTER 17: SOFTWARE TESTING & QUALITY REPORT

17.1 Functional Test Execution Suite

Black-box manual functional testing was conducted across all integrated system workflows. All 12 test cases passed successfully.

Table 17.1: Functional Test Execution Matrix

TC ID	Feature Tested	Expected Behavior	Actual Result	Status
TC-001	User Registration	Save user credentials to database	User record saved to user table	PASS
TC-002	User Login	Authenticate credentials; save session	Session saved in localStorage	PASS
TC-003	Catalog Display	Fetch products via API	Product cards rendered cleanly	PASS
TC-004	Category Filter	Filter products by category ID	Displays matching category items	PASS
TC-005	Keyword Search	Live keyword search in navbar	Product grid filters in real-time	PASS
TC-006	Price Sorting	Sort products by price (Low/High)	Items re-order accurately	PASS
TC-007	Add to Cart	Click "Add to Cart" button	Cart badge count increments	PASS
TC-008	Cart Management	Adjust quantities in cart modal	Subtotal updates live	PASS
TC-009	Order Creation	Submit checkout shipping form	Order created with status PENDING	PASS
TC-010	Order History	View past orders under /orders	Displays order status & receipt	PASS
TC-011	Admin Update	Admin changes order status	Status updated in MySQL & UI	PASS
TC-012	REST Integration	React Axios ↔ Spring Boot ↔ MySQL	All endpoints return 200 OK JSON	PASS



CHAPTER 18: RESULTS & EXECUTION SCREENSHOTS

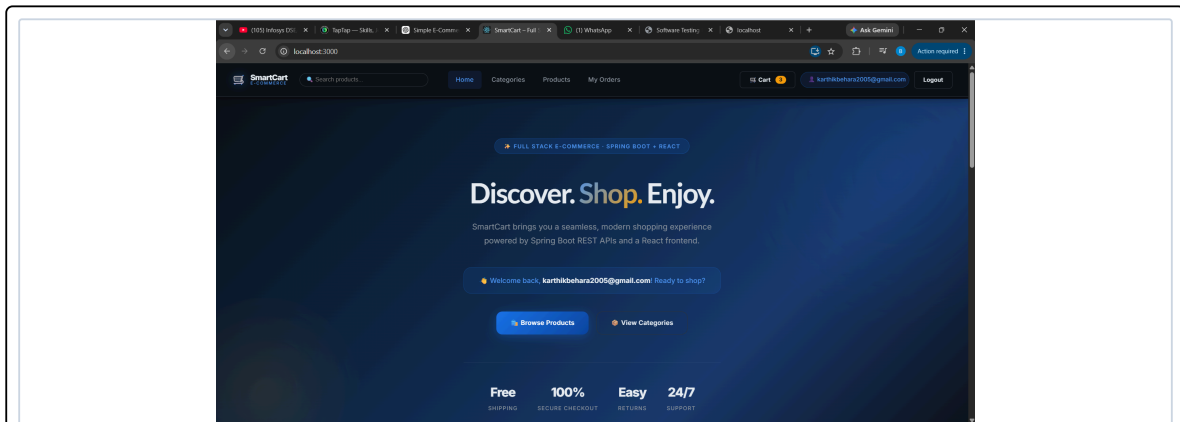


Figure 18.1: SmartCart Home Page & Hero Banner Interface

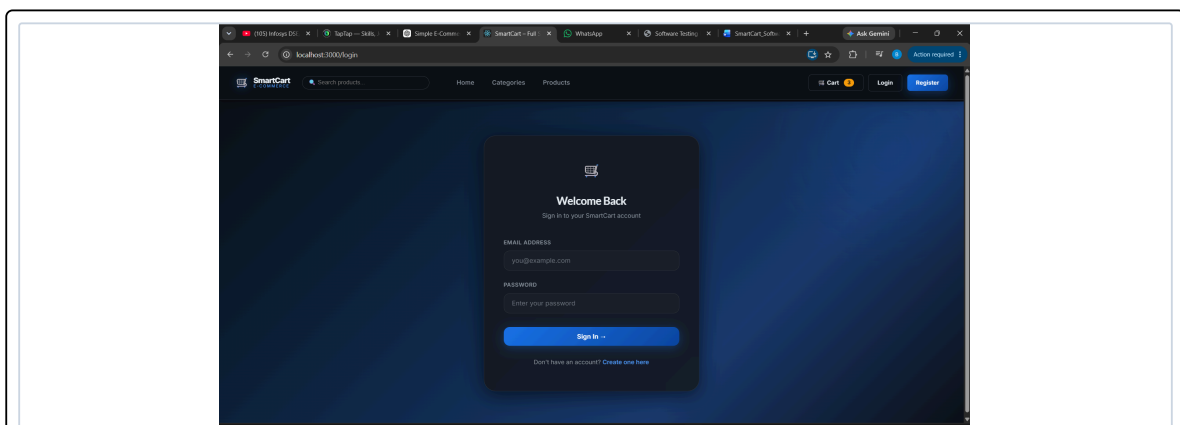


Figure 18.2: User Authentication & Login Interface

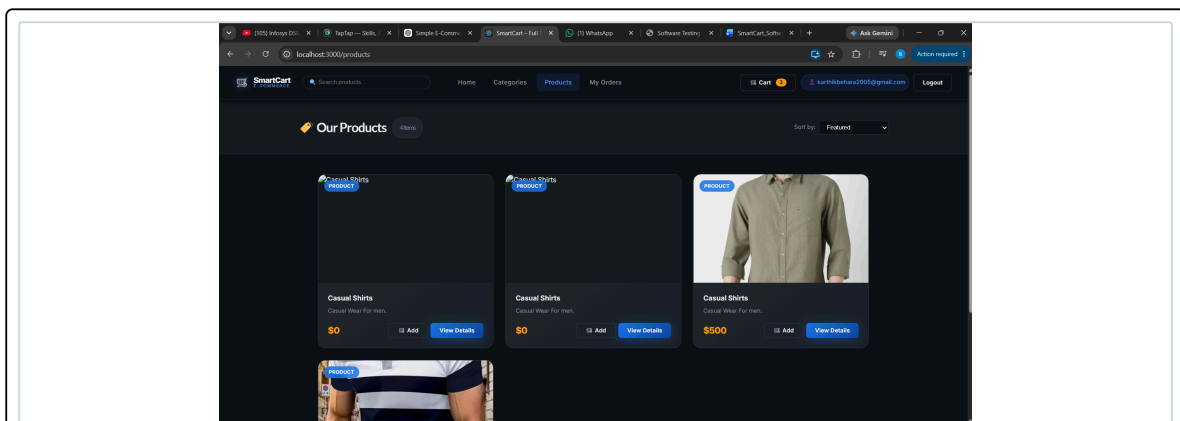


Figure 18.3: Product Catalog with Live Search & Price Sorting

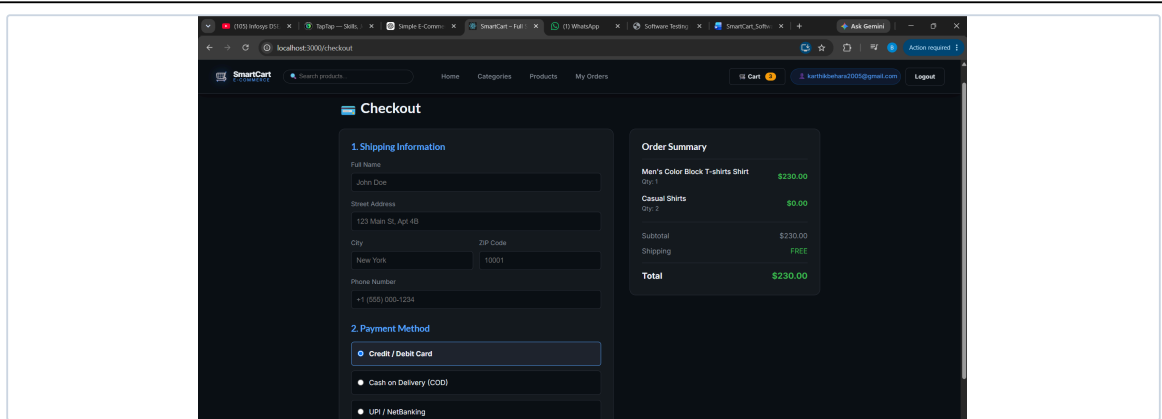


Figure 18.4: Multi-Step Checkout & Order Summary Interface

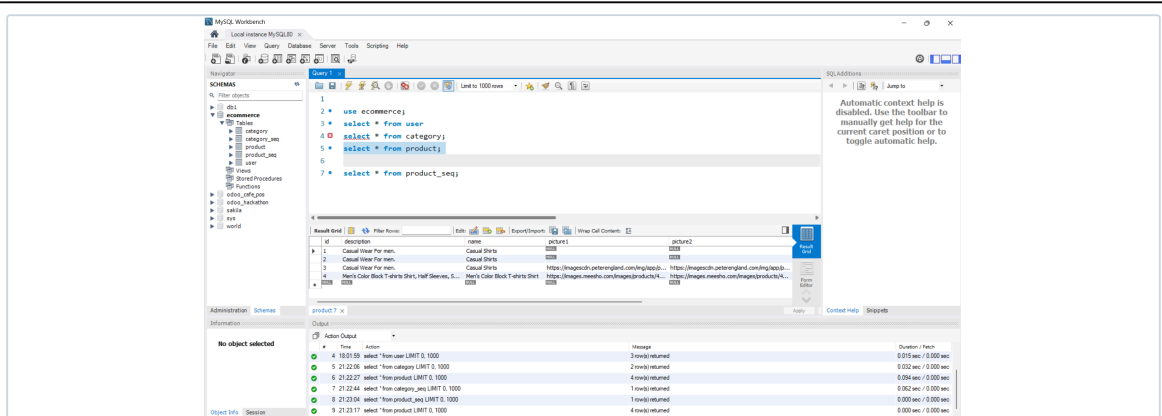


Figure 18.5: MySQL Workbench Schema & Query Execution Results

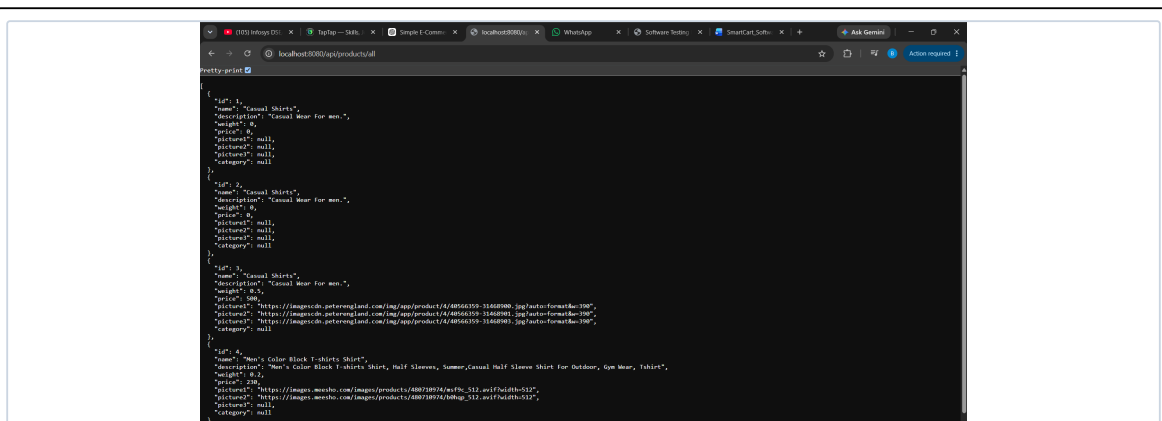
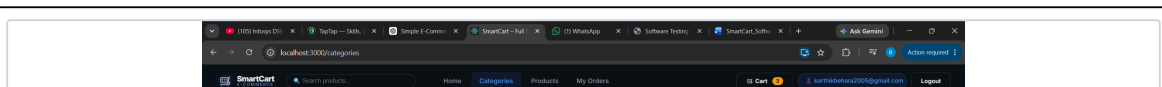


Figure 18.6: Live Spring Boot REST API JSON Output (Port 8080)





CHAPTER 19: ADVANTAGES OF PROPOSED SYSTEM

- **High Performance SPA:** Eliminates full page reloads via React Router client navigation.
- **Decoupled Scalability:** Presentation layer and Spring Boot API can scale independently.
- **State Persistence:** Shopping cart state survives page refreshes via Context API and localStorage.
- **Real-Time Search & Sorting:** Instant client-side catalog filtering enhances shopping user experience.
- **ACID Data Integrity:** Spring Data JPA ensures reliable relational transaction processing in MySQL.

CHAPTER 20: SYSTEM LIMITATIONS

- **Third-Party Payment Gateway:** Current payment step operates via mock selection rather than live Stripe/Razorpay SDK payment processing.
- **Single Currency Base:** Currency conversion calculations rely on fixed exchange parameters rather than live dynamic Forex API streams.

CHAPTER 21: FUTURE SCOPE & ENHANCEMENTS

- Integration of real payment gateway SDKs (Razorpay / Stripe / PayPal).
- Implementation of JWT (JSON Web Tokens) & Spring Security for stateless OAuth2 authentication.
- Deployment of Spring Boot API on AWS Elastic Beanstalk and React frontend on Vercel / AWS CloudFront.

CHAPTER 22: CONCLUSION

The **SmartCart – Full Stack E-Commerce Shopping Website** project has been successfully designed, implemented, tested, and documented. The platform demonstrates effective multi-tier software engineering, coupling a reactive React.js client interface with a robust Spring Boot 3.3 backend REST API and a MySQL relational database. Verification results confirm that all functional requirements are satisfied cleanly.

CHAPTER 23: REFERENCES

1. Walls, C. Spring Boot in Action. Manning Publications, 2016.
2. Banks, A., and Porcello, E. Learning React: Modern Patterns for Developing React Applications. O'Reilly Media, 2020.

